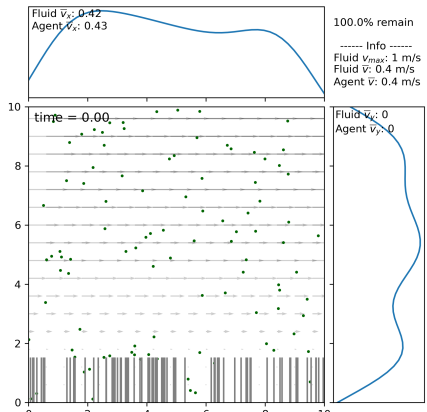
USING PLANKTOS

Agent-based transport and behavior in fluid environments

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part VI • Chapter 35



Planktos documentation image `docs/_static/basic_ex_2d_agents.png` (commit `a1c2128`, GPLv3).

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT YOU WILL BE ABLE TO BUILD

Core workflow

- create a 2D or 3D `Environment`;
- specify outer-domain boundary conditions;
- generate or import a velocity field and mesh;
- initialize one or more vectorized `Swarm`s;
- advance, inspect, plot, animate, and export.

Research extensions

- encode deterministic, stochastic, or social behavior;
- choose sliding, sticky, or disabled mesh collisions;
- compute KDE and finite-time Lyapunov exponent fields;
- reproduce a current Vicsek-model example;
- run the main ones on this series' IB2d and IBAMR flows.

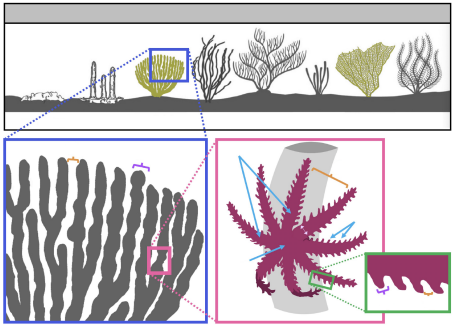
Every code fragment targets **Planktos v1.0.3**, repository commit `a1c2128` (2026-08-19). Unless a slide explicitly continues prior state, treat each fragment as a separate pattern in a fresh environment. The plots of these calls are this series' runs (`examples/Planktos`, slide 51); other pictures are credited.

MODELING SCOPE: AGENTS RESPOND TO A PRESCRIBED FLOW

Planktos is designed for point-particle organisms or particles whose feedback on the surrounding flow is negligible. A useful template is

$$d\boldsymbol{X} = \big[\boldsymbol{u}(\boldsymbol{X}, t) + \boldsymbol{b}(\boldsymbol{X}, t; \theta)\big]\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t,$$

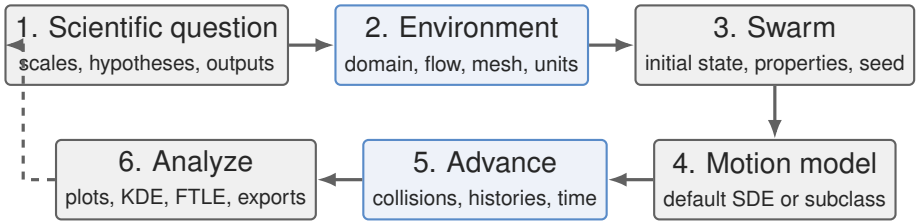
- $\boldsymbol{u}$: prescribed CFD or analytical velocity field;
- $\boldsymbol{b}$: swimming, settling, taxis, or social drift;
- $\boldsymbol{\sigma}$: stochastic variability, constant in space (it may still differ per agent);
- boundaries and immersed meshes constrain the proposed step.



Biological scales and gorgonian morphology: Hamlet et al. (2023), Fig. 1 (adapted from Kupfner Johnson & Hallock, 2020), CC BY 4.0.

A spatially varying $\boldsymbol{\sigma}(\boldsymbol{X})$ needs a Milstein-type scheme for strong order 1; the built-in Euler–Maruyama step is only order 0.5 in that case and does not support it directly. **One-way coupling.** Agent motion does *not* alter $\boldsymbol{u}$; use a coupled solver when feedback matters.

THE END-TO-END WORKFLOW



> Treat plotting and diagnostics as part of the simulation loop: inspect early, then scale up.

Schematic: instructor's course notes.

INSTALL FROM THE CLONED REPOSITORY

Clone the official repository, then make an isolated environment inside it.

```
git clone https://github.com/mountaindust/Planktos.git
cd Planktos
python -m venv .venv

# PowerShell
.\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install -e .
python -c "import planktos; print(planktos.__version__)"
```

Expected version for this deck: `1.0.3`.

Editable install

`-e` points the environment at the clone, so a later `git pull` updates the import without reinstalling.

Smoke test

Run `python examples/basic_ex_2d.py`; close each interactive plot to continue.

Adapted from the repository `README.md` Quickstart. Examples that read data files are run from `examples/`, as the documentation says.

DEPENDENCIES: WHAT IS INSTALLED, AND WHAT IS OPTIONAL

Declared core	Role
Python ≥ 3.8	runtime (development uses Python 3.12)
NumPy ≥ 1.19	vectorized agent and field arrays
SciPy $\geq 1.10.1$	interpolation, KDE, ODE integration
Matplotlib ≥ 3.0	plots and animations
pandas	per-agent properties
VTK/PyVista	≥ 9.2/ ≥ 0.44; CFD/mesh I/O and VTK output

Optional extras

```
python -m pip install -e
    ".[STL,netCDF,test]"
```

- `numpy-stl`: STL input;
- `netCDF4`: NetCDF input;
- `pytest`: test suite.

VTK installation

The project documentation highly recommends Miniforge (instead of Anaconda) and conda-forge for as much as possible, VTK and PyVista included.

Versions and extras are from `setup.cfg` at commit `a1c2128`; the development Python and the Miniforge advice from `docs/quickstart.rst`.

FFmpeg is needed only when saving animations

```
# Verify that the executable is on PATH
ffmpeg -version

# PowerShell can also locate it explicitly
Get-Command ffmpeg
```

Without FFmpeg

Simulations, static plots, and interactive animations still work. A filename passed to `plot_all` triggers the external movie writer.

If it is not found

1. install FFmpeg with the platform's package manager or official installer;
2. add its executable directory to `PATH`;
3. restart the terminal or IDE;
4. rerun `ffmpeg -version` in the same environment that runs Python.

> For long simulations, save every k-th state as a frame; rendering every time step is rarely useful.

RUNNING THE TESTS

```
cd Planktos                          # the clone's base directory
python -m pip install pytest
pytest                               # the test suite
pytest --runslow                     # also the parallelization tests and the plotting smoke tests
```

- The 2021 deck: “All tests can be run by typing pytest into a terminal in the base directory.” It is the same in 2026, and GitHub Actions run the suite and a spell check on pushes to the main branches and on pull requests (“with every push”, say the 1.0.1 notes), with Python 3.8 and 3.12.
- 1.0.2 grew the suite: the immersed-boundary collision code is covered to 99 %, with checks that do not depend on the geometry (no penetration, no amplification of motion, independence from position, axis orientation and units).
- Not run for this chapter: the package here was installed with `pip install git+...`, which brings no `tests` folder. The scripts of `examples/Planktos` (slide 51) ran instead.

Planktos `README.md` (“Tests”; October 2026) and release notes v1.0.1, v1.0.2; “Getting started with Planktos” (Miller lab, 2021), slide 12.

INSTALLING, 2021 AND 2026

the lab's 2021 deck ("Getting started with Planktos")	Planktos 1.0.3 (2026)
Python 3.5+	Python ≥ 3.8 (tested on 3.8 and 3.12)
download with git or GitHub Desktop, and pull often ("Christopher just fixed some bugs today")	`git clone`, then `pip install .` or `pip install -e .`; pull often
NumPy, SciPy, Matplotlib 3.x, pandas	NumPy ≥ 1.19, SciPy $\geq 1.10.1$ ("earlier versions have a broken interpn"), Matplotlib ≥ 3.0, pandas
FFmpeg from conda-forge (`conda install -c conda-forge ffmpeg`)	FFmpeg from anywhere, on the `PATH` (slide 7)
VTK and PyVista from conda-forge, installed with mamba	VTK ≥ 9.2, PyVista ≥ 0.44; conda-forge and mamba still advised
numpy-stl for STL files	numpy-stl $\geq 2.16.3$; netCDF4 $\geq 1.5.7$ for NetCDF
pytest for the tests	the same, and `--runslow` (slide 8)
IBAMR data: "a Python 2.7 environment with numpy and VisIt installed (VisIt's Python API is written in Python 2.7)"	VisIt's Python is 3 from VisIt 3.2 on (chapter 32): chapter 33's `resample_export.py`, then `vtk_for_planktos.py` (slide 42)

"Getting started with Planktos" (Miller lab, 2021), slides 8–11; Planktos `README.md` (October 2026); chapter 32.

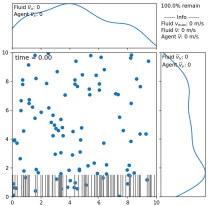
THE 2021 EXAMPLES, WITH TODAY'S NAMES

2021	what it showed	in 1.0.3 (`examples/`)
`example_2d.py`	time-varying Brinkman flow across a porous layer	`basic_ex_2d.py` (Brinkman flow; the start of the tour)
`example_3d.py`, `example_3d_flat.py`	the same in 3D; "Same as previous but different dimensions in each direction"	`basic_ex_3d.py` (a $20 \times 10 \times 4$ box)
`example_IBAMR.py`	3D flow around cylinders read from IBAMR	`ex_IBAMR_ibmesh.py`: IBAMR test data (dumps 3–5), a cylinder by convex hull, the domain tiled the same file
`example_IBAMR_ibmesh.py`	agents around a cylinder in 3D, saved as a movie	
`example_IBAMR_ind_var.py`	a 2D Brinkman layer, agents that differ	`ex_ind_var.py` (`example_ind_var.py` in the 2021 repository): the covariance differs from agent to agent
an IB2d leaf (not on GitHub then)	IB2d flow past a leaf made from a vertex file	no leaf; `ex_ib2d_ibmesh.py` reads IB2d's flow around a cylinder (data from a link)
a COMSOL sea fan (not on GitHub then)	COMSOL flow through an STL sea fan	`ex_sticky_seafan_3d.py`: a COMSOL cylinder tiled into an array, sticky (data from a link)

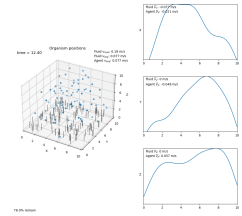
New since 2021: `ex_agent_behavior.py`, `ex_ode_gen.py`, `ex_poisson_search.py`, `ex_produce_ftle_2d.py`, `ex_ib2d_sticky.py`, `ex_ib2d_mvbnd_sticky.py`, `ex_vicsek_model_2d.py`, `ex_vicsek_model_3d.py`. `old_examples/` keeps `old_example_2d.py`, `old_example_3d.py`, `old_ex_pltcyl.py`.

"Getting started with Planktos" (Miller lab, 2021), slides 13–20; the repository's `examples/` folder and the files' docstrings (GitHub, 17 October 2026; 2021: commit `58cca0d`). The repository of June 2021 also had `example_2d_plankton_agent.py`, `example_ib2d_ibmesh.py`, `plt_cyl.py` and `basic_brine_shrimp/`, which the 2021 deck did not show.

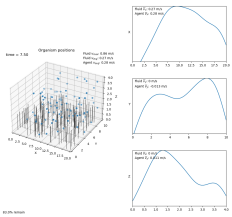
THE 2021 EXAMPLES' OUTPUT (1/2)



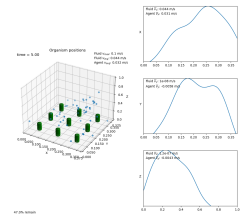
`example_2d.py`: Brinkman flow across a porous layer



`example_3d.py`: the same in 3D



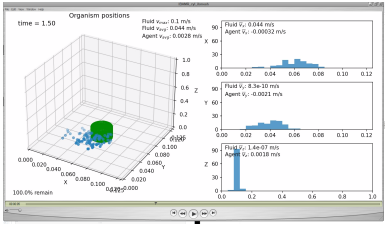
`example_3d_flat.py`: different lengths in each direction



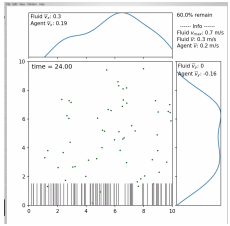
`example_IBAMR.py`: IBAMR flow around cylinders

Planktos output in "Getting started with Planktos" (Miller lab, 2021), slides 13–16.

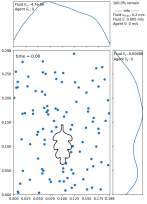
THE 2021 EXAMPLES' OUTPUT (2/2)



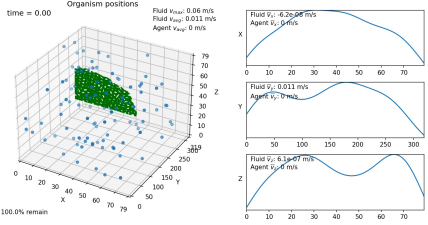
`example_IBAMR_ibmesh.py`: a frame of its movie



`example_IBAMR_ind_var.py`: agents that differ



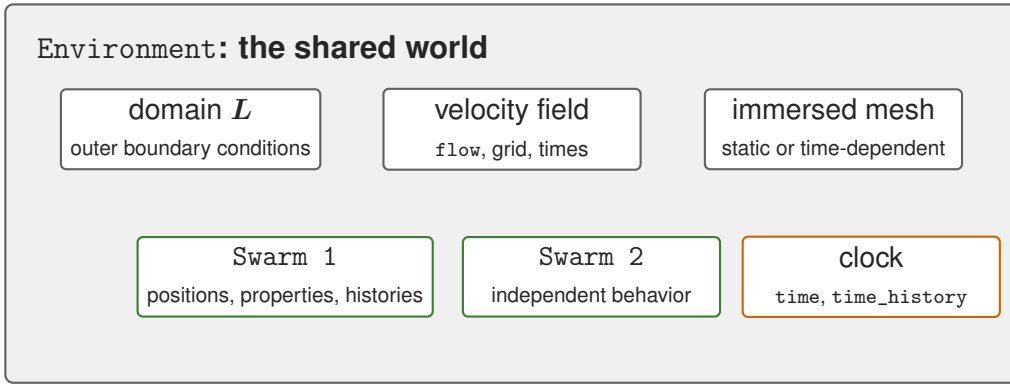
IB2d flow past a leaf (static mesh from a vertex file)



COMSOL flow through an STL sea fan

Planktos output in "Getting started with Planktos" (Miller lab, 2021), slides 17–20.

TWO OBJECTS ORGANIZE ALMOST EVERYTHING



- A swarm is a vectorized group: agents are rows of NumPy arrays, not separate Python objects.
- All swarms in one environment sample the same flow and mesh.

Schematic: instructor's course notes; object model from `planktos/_environment.py` and `_swarm.py`, commit `a1c2128`.

CREATE THE FIRST ENVIRONMENT

```python
import planktos

env = planktos.Environment(
    Lx=20, Ly=5,
    x_bndry="zero",
    y_bndry=["noflux", "noflux"],
    rho=1000, mu=1.0,
    units="m")

print(env.L)        # [20, 5]
print(env.nu)       # mu/rho = 0.001
print(env.time)     # 0.0
```

Defaults worth knowing

- 10×10 two-dimensional domain;
- zero velocity everywhere;
- `zero` boundaries in x, y (`noflux` in z when `Lz` is given);
- no swarms or immersed mesh;
- `time = 0.0`.

Units are metadata

`units` does not rescale data automatically. Make the flow, mesh, agent parameters, gravity, and time convention consistent yourself.

OUTER-DOMAIN BOUNDARY CONDITIONS

Setting	What happens to an agent crossing the box	Typical use
`zero`	The agent is masked and no longer updated or plotted.	open/outflow boundary
`noflux`	The proposed displacement through the wall is projected along it.	solid outer wall
`periodic`	The agent exits one side and re-enters on the opposite side.	repeating domain

Each axis accepts one string for both sides or a two-entry sequence:

```
x_bndry=["zero","noflux"].
```

The order is low-side, high-side: left/right, bottom/top, and low/high z.

Do not conflate two layers

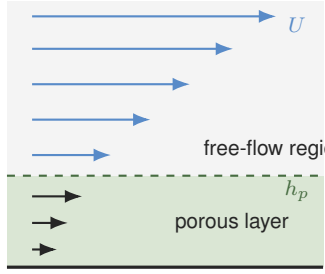
These settings govern the rectangular domain. Immersed-mesh collisions are controlled separately by `ib_condition` or `ib_collisions`.

GENERATE A BRINKMAN-LAYER FLOW

```
env = planktos.Environment(
    Lx=20, Ly=5, rho=1000, mu=1000)

env.set_brinkman_flow(
    alpha=66, h_p=1.5,
    U=1, dpdx=1, res=101)

env.plot_flow()
```



Code adapted from `examples/basic_ex_2d.py`. Schematic: instructor's course notes.

- `alpha`: inverse hydraulic-permeability parameter (`alpha` $\to 0$ is free flow; `alpha=0` itself stops with `ZeroDivisionError`);
- `h_p`: porous-layer height;
- `U`: velocity at the top of the domain;
- `dpdx`: pressure-gradient parameter;
- `res`: points across the resolved profile.

The profile is the analytic, steady, fully developed solution of Brinkman's equation (Brinkman 1949; Strickland et al. 2017), with a Darcy drag $-\alpha\mu\boldsymbol{u}$ in the porous layer per the docstring; the closed-form solution uses $e^{-\alpha h_p}$, so check the units of `alpha` (inverse length there) before reusing values. `mu` and `rho` must be set, or the call raises.

READ THE FIELD REPRESENTATION BEFORE MOVING AGENTS

```
print(len(env.flow))
print([a.shape for a in env.flow])
print(env.flow_points)
print(env.flow_times)
print(env.L, env.units)
```

Array convention

- static: `(nx, ny[, nz])`;
- time-dependent: `(nt, nx, ny[, nz])`;
- one array for each velocity component;
- `flow_points[i]` indexes spatial axis i.

Interpolation contract

Planktos uses linear spatial interpolation on a rectilinear grid and cubic splines in time for time-dependent data.

Preflight checks

- component shapes agree;
- coordinate arrays increase and match the axes;
- domain boundaries and mesh coordinates align;
- `flow_times` spans the intended run;
- speeds and units are physically plausible.

SUPPLY A HAND-BUILT RECTILINEAR VELOCITY FIELD

```python
import numpy as np
import planktos

x = np.linspace(0, 2, 81)
y = np.linspace(0, 1, 41)
X, Y = np.meshgrid(x, y, indexing="ij")

u = 4*Y*(1-Y)          # streamwise component
v = np.zeros_like(u)

env = planktos.Environment(
    Lx=x[-1], Ly=y[-1], flow=[u, v])
env.flow_points = (x, y)
env.plot_flow()
```

Why `indexing="ij"`?

Axis 0 is x, axis 1 is y. This matches the current Planktos storage convention and avoids a silent transpose.

Time-dependent fields

Stack snapshots on axis 0 and pass `flow_times` to the constructor. The remaining axes stay $(x, y[, z])$.

Coordinates

Set `flow_points` explicitly when the coordinates are not the constructor's uniform grid.

Convention cross-checked against current flow-generation and analysis tests.

ADD A VECTORIZED SWARM

```
# Convenience method: automatically uses env
s = env.add_swarm(
    swarm_size=200,
    init="random", seed=2026,
    name="larvae", color="darkgreen")

# Equivalent explicit construction
s2 = planktos.Swarm(
    swarm_size=50, envir=env,
    init=(0.1, 0.5), seed=7)

print(s.N, s.positions.shape)
print(len(env.swarms))
```

State layout

- `positions`: masked $N \times d$ array;
- `velocities`: $N \times d$;
- `accelerations`: $N \times d$;
- `props`: per-agent DataFrame;
- `shared_props`: shared dictionary.

> Vectorization is the design choice: write array operations over agents whenever possible.

CHOOSE INITIAL POSITIONS DELIBERATELY

```
# Uniform random over the domain
s_random = env.add_swarm(500, init="random", seed=1)

# One shared starting point
s_point = env.add_swarm(100, init=(0.15, 0.50), seed=2)

# Explicit N-by-d array
rng = np.random.default_rng(3)
xy = np.column_stack((
    rng.uniform(0.05, 0.15, 300),
    rng.uniform(0.20, 0.80, 300)))
s_patch = env.add_swarm(init=xy, seed=3)

# Grid, optionally excluding closed mesh interiors
s_grid = env.add_swarm(
    init="grid", grid_dim=(80, 40), testdir="x1")
```

Interpretation

- `random`: a uniform domain sample;
- point: a release cohort;
- $N \times d$ array: measured or designed initial condition (its row count sets N; `swarm_size` is ignored);
- `grid`: transport-map or FTLE seed lattice.

Grid masking

`testdir` needs a loaded immersed mesh: without one the call above stops (“AttributeError: ’NoneType’ object has no attribute ’reshape”’), so leave it out then. With a closed mesh, the chosen ray direction must give a robust inside/outside test.

SHARED PROPERTIES, INDIVIDUAL VARIATION, AND SEEDS

```python
import numpy as np

# Same for every agent
s.shared_props["mu"] = np.array([0.0, 0.0])
s.shared_props["cov"] = np.eye(2)*(0.01**2)
s.shared_props["diam"] = 1.0e-3

# One value per agent
rng = np.random.default_rng(2026)
s.props["swim_speed"] = rng.lognormal(
    mean=np.log(0.02), sigma=0.15, size=s.N)

# Uniform access: scalar/array as appropriate
speed = s.get_prop("swim_speed")
diam = s.get_prop("diam")
```

The two random streams

- `seed=` initializes `s.rndState`, used by built-in stochastic motion.
- A separate NumPy generator should seed any user-created initial conditions or properties.

Covariance units

The default SDE uses step covariance `cov` dt. Thus `cov` has units of length^2/time, not length^2 per step.

THE DEFAULT MODEL IS EULER–MARUYAMA DRIFT–DIFFUSION

For each active agent, the default `apply_agent_model` approximates

$$d\boldsymbol{X} = [\boldsymbol{u}(\boldsymbol{X}, t) + \boldsymbol{\mu}]\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t, \qquad \mathrm{cov} = \boldsymbol{\sigma}\boldsymbol{\sigma}^T = 2\boldsymbol{D}.$$

One step is

$$\boldsymbol{X}^{n+1,*} = \boldsymbol{X}^n + [\boldsymbol{u}(\boldsymbol{X}^n, t^n) + \boldsymbol{\mu}]\, \Delta t + \mathcal{N}(\boldsymbol{0}, \mathrm{cov}\, \Delta t)\,,$$

followed by outer-boundary and immersed-mesh handling.

Deterministic tracer

For spatial dimension d, set `mu = np.zeros(d)` and `cov = np.zeros((d,d))`; then motion is forward Euler using the local fluid velocity.

Time-step audit

Check both temporal convergence and collision resolution. A step can be statistically reasonable yet too large to resolve a narrow obstacle.

Adapted from the `Swarm.apply_agent_model` and `motion.Euler_brownian_motion` docstrings (commit `a1c2128`); scheme: Kloeden & Platen (1992), §10.2. A `'D'` property (converted as $\mathrm{cov} = 2D$) is read only when there is no `'cov'`, and every Swarm starts with `cov` = the identity unless given one: delete it first (`del s.shared_props['cov']`).

ADVANCE THE SWARM AND READ ITS HISTORIES

```python
import numpy as np

dt = 0.1
for _ in range(round(24.0/dt)):
    s.move(dt, silent=True)

print(env.time)
print(len(env.time_history))
print(len(s.full_pos_history))

# Current diagnostics
mask = np.ma.getmaskarray(s.positions)
active = ~np.all(mask, axis=1)
print(active.sum(), "agents remain active")
print(s.velocities.mean(axis=0))
```

What `move` manages

1. calls `apply_agent_model(dt)`;
2. stores prior position/velocity/property state;
3. estimates velocity and acceleration;
4. applies boundaries and mesh collisions;
5. calls `after_move(dt)`;
6. advances environment time.

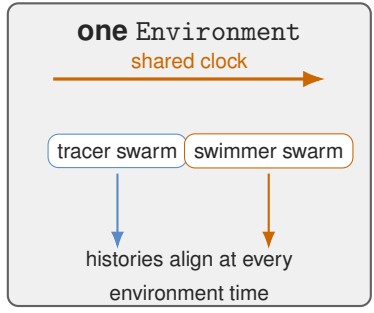
History note: `full_pos_history` and `full_vel_history` return stored history plus the current state without mutating it.

MOVE MULTIPLE SWARMS ON ONE SHARED CLOCK

```
tracers = env.add_swarm(
    500, seed=10, name="tracers", color="navy")
swimmers = env.add_swarm(
    500, seed=11, name="swimmers", color="orange")
swimmers.shared_props["mu"] = np.array([0.02, 0.0])

for _ in range(400):
    env.move_swarms(
        dt=0.05,
        ib_collisions="sliding",
        silent=True)
```

Default differs: `move_swarms` uses `ib_collisions="sliding"` unless told otherwise, overriding each swarm's `ib_condition`; pass `ib_collisions="default"` to honor each swarm's own setting.



Avoid double time advancement

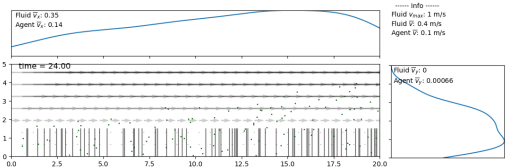
Do not call `s1.move(dt)` and then `s2.move(dt)` when both should occupy the same time step. Use `env.move_swarms`.

Schematic: instructor's course notes.

MAKE AN EARLY DIAGNOSTIC PLOT

```
# Current state with velocity arrows
s.plot(
    fluid="quiver",
    dist="density",
    circ_rad=0.08,
    plot_heading=False)

# Closest stored state to t=8.0
s.plot(
    t=8.0,
    filename="snapshot.png",
    fluid="vort",
    save_kwargs={"dpi": 200})
```



The first call above, run: `examples/Planktos/brinkman_first.py` runs most of this chapter's first-run fragments, in order (Planktos 1.0.3): 200 agents in the Brinkman flow at $t = 24$ s, 110 of them still in the box (the default `zero` boundaries remove the rest). The documentation's own render of this example (`docs/_static/basic_ex_2d_agents.png`, GPLv3) is on the title slide.

2D fluid overlays

`fluid="quiver"` shows velocity vectors; `fluid="vort"` shows scalar vorticity. These overlays are 2D-only.

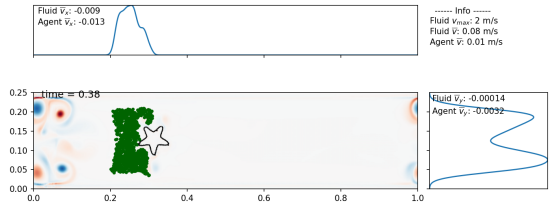
KDE TURNS PARTICLES INTO A DISTRIBUTION DIAGNOSTIC

```python
# Gaussian KDE with Scott's factor
s.plot(dist="density", fluid="vort")

# Covariance-based width
s.plot(dist="cov", fluid="vort")

# User bandwidth factor or a histogram
s.plot(dist=0.35)
s.plot(dist="hist")
```

- Compare concentration shape, tails, and obstacle wakes.
- Report the bandwidth rule with any quantitative comparison.
- Check sensitivity to sample size and boundary truncation.
- A KDE is a smoothed estimator, not a separately solved concentration field.



`dist="density"` with `fluid="vort"` on this series' flow: 1000 agents released just upstream of chapter 4f's starfish, at $t = 0.38$ s; the KDE of y has a peak on each side of the starfish (`examples/Planktos/starfish_kde.py`). Strickland et al. (2022), Fig. 4, shows the same kind of plot for an IB2d cylinder flow (not reproduced: the article is not openly licensed).

ANIMATE SELECTIVELY; EXPORT ANALYSIS-READY DATA

```
# Use every fifth stored state
frames = range(0, len(s.full_pos_history), 5)
s.plot_all(
    movie_filename="trial_01.mp4",
    frames=frames, fps=10,
    fluid="quiver", plot_heading=False)

# Data products
s.save_data("results", "trial_01")
s.save_pos_to_csv("results/trial_01_positions.csv")
s.save_pos_to_vtk("results", "trial_01")
```

What `save_data` writes: CSV files for the position history (with mask and time stamps) and for the current velocities and accelerations; `props` to JSON; `shared_props` to .npz. `props_history` is not saved. `save_pos_to_vtk` writes one PolyData file per stored time. Why `quiver`: on a static flow such as this Brinkman layer, `plot_all(fluid="vort")` stops in 1.0.3 ("ValueError: not enough values to unpack (expected 2, got 1)") before a frame is drawn (slide 53).

Calls match current `Swarm.plot_all` and save-method signatures.

Choose the product by use

- `save_data`: position history plus current kinematics and properties (file formats at left);
- CSV: portable position history;
- VTK: ParaView/VisIt workflow;
- MP4: communication, not raw data.

Cost control

Use `frames` for temporal subsampling and `downsamp` for display-only agent subsampling. Statistics still use the full population.

CUSTOM BEHAVIOR: OVERRIDE ONE HOOK, NOT THE HARNESS

```python
class MySwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        # Compute proposed N-by-d positions.
        new_positions = ...
        return new_positions
```

Contract

- keep the signature `(self, dt)`;
- return an $N \times d$ array;
- do not assign `positions`, `velocities`, or `accelerations`;
- let `move` record history and apply collisions.

Current contract: `planktos/_swarm.py` and `examples/ex_agent_behavior.py`.

Current versus historical API

The 2022 paper describes a historical override named `get_positions`. The current v1.0.3 repository and examples use `apply_agent_model(self, dt)`.

> Use `after_move(dt)` for state changes based on the accepted, collision-corrected endpoint.

EXAMPLE: FLUID ADVECTION PLUS DIRECTED SWIMMING

```python
import numpy as np
import planktos

class TargetSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        target = np.array([1.8, 0.5])
        direction = target - self.positions
        norm = np.linalg.norm(direction, axis=1)[:, None]
        direction = direction/np.maximum(norm, 1.0e-12)

        speed = self.get_prop("swim_speed")[:, None]
        velocity = self.get_fluid_drift() + speed*direction
        return self.positions + velocity*dt
```

Pattern adapted from `examples/ex_agent_behavior.py`.

Useful field queries

- `get_fluid_drift()`;
- `get_dudt()`;
- `get_fluid_mag_gradient()`;
- `get_DuDt()`.

Numerical hygiene

Guard normalizations, preserve masks, and call expensive interpolants once per time step rather than once per agent.

DETERMINISTIC ODE MOTION WITH THE RK45 WRAPPER

```python
import planktos
from planktos import motion

class AccurateTracer(planktos.Swarm):
    def apply_agent_model(self, dt):
        ode = motion.tracer_particles(
            self, incl_dvdt=False)
        t0 = self.envir.time
        # positions is a masked array; strip masked
        # (exited) rows before RK45, as calculate_FTLE does
        return motion.RK45(
            ode, t0, self.positions.copy(), t0 + dt,
            rtol=1.0e-6, atol=1.0e-9)
```

About the comment: for brevity the listing passes the whole masked array. SciPy drops the mask, so exited rows are integrated from stale coordinates. Strip them as `calculate_FTLE` does, `y = pos[~mask[:,0]]`, and scatter the result back. (`incl_dvdt=False` makes the tracer ODE act on $N \times d$ positions only.)

Current solver contract

`motion.RK45` wraps SciPy `solve_ivp`; the right-hand side has signature `fun(t, x)` and returns the same 2D shape as `x`.

Two time scales

The internal ODE solver adapts substeps, but Planktos applies boundary and mesh collision handling only after the outer `move(dt)` proposal.

COMBINE AN ODE DRIFT WITH STOCHASTIC MOTION

```python
import numpy as np
import planktos
from planktos import motion

def social_bias(pos):
    return 0.1*(pos.mean(axis=0) - pos)

def behavior_ode(swarm):
    def rhs(t, x):
        N = x.shape[0]//2
        vel = swarm.get_fluid_drift(t, x[:N])
        vel += social_bias(x[:N])
        return np.concatenate((
            vel, np.zeros((N, x.shape[1]))))
    return rhs

class SocialSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        return motion.Euler_brownian_motion(
            self, dt, ode=behavior_ode(self))
```

Adapted from `examples/ex_ode_gen.py`.

State convention

An ODE passed to `Euler_brownian_motion` receives $2N \times d$: positions followed by velocities. It returns their time derivatives in the same layout.

Fluid drift is explicit

When an `ode` is supplied, include fluid interaction in that ODE if it is part of the model; it is not added automatically there.

The swarm's `mu` property is still added to the ODE's velocity rows (zero by default), and the step is the Taylor update $\Delta \boldsymbol{x} = \boldsymbol{v}\,\Delta t + \frac{1}{2}\boldsymbol{a}\,\Delta t^2$ plus the `cov` noise.

BUILT-IN INERTIAL-PARTICLE DYNAMICS (1/2)

Set ρ, μ, ν in the constructor: the constructor computes the third value correctly, but in v1.0.3 the `nu` and `rho` property setters compute $\mu = \nu/\rho$ instead of $\mu = \nu\rho$ when μ is unset (an upstream bug, still present on `master` as of October 2026; not yet filed as a GitHub issue).

```
import numpy as np
import planktos
from planktos import motion

zero = np.zeros((3, 3))
env = planktos.Environment(
    Lx=1, Ly=1, rho=1000, nu=1e-6,
    char_L=0.01, U=0.1,
    flow=[zero, zero.copy()])
```

The listing continues on the next slide with the Swarm subclass. Here $\mathrm{Re} = U\,$`char_L`$/\nu = 0.1 \cdot 0.01/10^{-6} = 1000$, and the constructor sets $\mu = \nu\rho = 10^{-3}$.

More on the setters: once all three are set, a setter only warns and leaves the other two unchanged. The bug is still on `master` (last commit 2026-09-16); no public GitHub issue for it was found as of 2026-10-17.

Required environment data

A velocity field (explicit zeros are valid), `char_L`, `U`, `nu`, and gravity; fluid density unless `R` is supplied.

Required particle data

Density ratio `R` (or particle `rho`) and diameter `diam`.

Model regime

The implementation is the linearized Maxey–Riley form documented for $R/\mathrm{St} \gg 1$; verify that approximation for the particles and flow under study. Here, the displayed parameters (with `R` and `diam` from slide 2/2) give $R/\mathrm{St} \approx 120$.

Setter behavior: `Environment.rho/mu/nu` in `planktos/_environment.py`, commit `a1c2128`.

BUILT-IN INERTIAL-PARTICLE DYNAMICS (2/2)

```
class InertialSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        ode = motion.inertial_particles(self)
        # positions is a masked array; strip masked
        # (exited) rows before RK45, as calculate_FTLE does
        # NB: move() overwrites velocities with
        # (x_new-x_old)/dt; persist velocity in props
        # for a true second-order model
        state = np.ma.concatenate(
            (self.positions, self.velocities))
        out = motion.RK45(
            ode, self.envir.time, state,
            self.envir.time + dt)
        return out[:self.N]

s = InertialSwarm(envir=env, seed=4)
s.shared_props.update({"R": 2/3, "diam": 1e-4})
```

Units: Planktos applies the dimensionless equation at right to the data's metres and seconds, so the slip here decays at $R/\mathrm{St} = 120$ s^{-1} (measured), while the physical rate is $(R/\mathrm{St})\,U/\texttt{char_L} = 3\nu/a^2 = 1200$ s^{-1}: use units with $U = \texttt{char_L}$, or rescale.

Model: `motion.inertial_particles` docstring, after Maxey & Riley (1983) and Haller & Sapsis (2008).

The ODE being integrated

With state $(\boldsymbol{x}, \boldsymbol{v})$ stacked as $2N \times d$,

$$\dot{\boldsymbol{v}} = \tfrac{3R}{2}\tfrac{D\boldsymbol{u}}{Dt} - \tfrac{R}{\mathrm{St}}(\boldsymbol{v} - \boldsymbol{u}) + \big(1 - \tfrac{3R}{2}\big)\boldsymbol{g},$$

$$R = \frac{2\rho_f}{\rho_f + 2\rho_p},\ \mathrm{St} = \tfrac{2}{9}\big(\tfrac{a}{L}\big)^2\mathrm{Re},$$
$$a = \texttt{diam}/2.$$

Check:
$\mathrm{St} = \frac{2}{9}(5 \times 10^{-3})^2 \cdot 1000 \approx 5.6 \times 10^{-3}$, so $R/\mathrm{St} \approx 0.667/0.0056 \approx 120$. $R = 2/3$ is neutrally buoyant, so the gravity term vanishes.

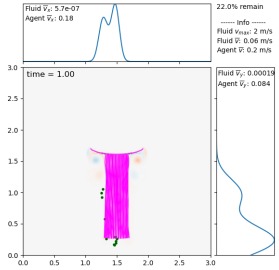
IMMERSED-MESH COLLISION MODES

Mode	Proposed step after a mesh hit
`sliding`	Project motion along the boundary, recursively using remaining displacement.
`sticky`	Stop at the first point of intersection for that time step.
`None`	Ignore immersed meshes.
`'default'`	Use the Swarm's `ib_condition` (default `'sliding'`).

Set a swarm default with `ib_condition`; override per step with

```
s.move(dt, ib_collisions="sticky").
```

`s.ib_collision_idx` is a per-agent array of the last-step element index hit, -1 if none: precisely, the element of the *first* intersection during the most recent step (a sliding agent may go on to touch others). It is reset every step for agents still in the domain, so a captured agent held in place reads -1 on later steps; keep capture in `props`.



Sticky agents and a moving IB2d boundary; Planktos documentation image `docs/_static/ib2d_mvbnd_sticky.png` (commit `a1c2128`, GPLv3).

Geometry restriction

Avoid immersed structures that intersect periodic box boundaries; that interaction is not implemented in the current API.

One more case: a hit followed in the same step by a periodic or `noflux` box edge is lost from `ib_collision_idx`: the re-check at the edge resets it (to -1 if nothing more is hit).

MAKE “CAPTURED” A PERSISTENT BIOLOGICAL STATE

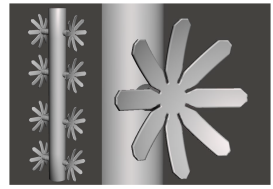
```python
import numpy as np
import planktos
from planktos import motion

class CaptureSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        proposed = motion.Euler_brownian_motion(self, dt)
        held = self.get_prop("captured")[:, None]
        return np.ma.where(held, self.positions, proposed)

    def after_move(self, dt):
        hit = self.ib_collision_idx >= 0
        self.props.loc[hit, "captured"] = True
        self.props.loc[hit, "color"] = "gold"

s = CaptureSwarm(
    envir=env, ib_condition="sticky",
    store_prop_history=True)
s.props["captured"] = np.zeros(s.N, dtype=bool)
s.props["color"] = np.full(s.N, s.shared_props["color"])
```

Pattern adapted from `examples/ex_sticky_seafan_3d.py`.



Cylinder and polyp geometry: Hamlet et al. (2023), Fig. 6A, CC BY 4.0.

Why both hooks?

`after_move` records new contacts after collision correction. `apply_agent_model` prevents previously captured agents from leaving on later steps.

IMPORT AN IB2D VELOCITY FIELD AND 2D MESH

```
env = planktos.Environment()

# From the repository root, first follow
# examples/ib2d_data/README.txt to download the data.
env.read_IB2d_fluid_data(
    "examples/ib2d_data",
    dt=5.0e-5, print_dump=1000)

env.read_IB2d_mesh_data(
    "examples/ib2d_data/channel.vertex",
    method="proximity")

env.plot_envir()
s = planktos.Swarm(
    envir=env, init=(0.1*env.L[0], 0.5*env.L[1]),
    seed=1)
```

Time mapping

Successive printed flow snapshots are separated by

$$\Delta t_{\text{data}} = \texttt{dt} \times \texttt{print_dump} = 0.05.$$

Mesh association

`proximity` connects nearby vertices. It can infer resolution from loaded flow or accept `res=` explicitly.

Inspect connectivity. Crossing segments without a shared vertex can create collision leaks. Always call `plot_envir()`.

Adapted from `examples/ex_ib2d_ibmesh.py`; the external dataset is linked from `examples/ib2d_data/README.txt`.

THIS SERIES' IB2D OUTPUT (1/2): READING CHAPTER 4F'S STARFISH RUN

```
env = planktos.Environment(x_bndry="periodic",
    y_bndry="noflux", rho=1000, mu=0.1, units="m")
env.read_IB2d_fluid_data("Starfish_Channel/viz_IB2d",
    dt=5e-5, print_dump=400, d_start=1)
print(env.L, env.flow_times[[0, -1]],
      env.fluid_domain_LLC)
# [0.9999974399999999, 0.24999936] [0.   0.38] (0.0,
    0.0)
```

Start at dump 1

pyIB2d writes dump 0's velocity transposed: `u.0000.vtk` and `uX.0000.vtk` say `DIMENSIONS 128 512 1`, every later dump `512 128 1` (`IBM_Driver.py` passes `U, V` there and `U.T, V.T` afterwards; `Omega` and `uMag` are right). From dump 0 the reader stops: "all the input array dimensions . . . must match".

- The fluid is at rest in dump 0, so only the run's first 0.02 s (the pulse starting from rest) is lost; Planktos's clock starts at the first dump it reads: Planktos's t is the run's $t - 0.02$ s. IB2d's issue #12 reports the same transposed dump 0 for the MATLAB version.
- `L` is 0.99999744, not 1: IB2d writes the grid spacing with six digits (`SPACING 0.00195312`).
- `rho` and `mu` go to the constructor (slide 32). IB2d is periodic, and the reader always adds the periodic end points (its docstring's `auto_regrid` is not a parameter).

`examples/Planktos/starfish_env.py`; https://github.com/nickabattista/IB2d/issues/12 (open, 17 October 2026).

THIS SERIES' IB2D OUTPUT (2/2): JOINING THE STARFISH'S OUTLINE

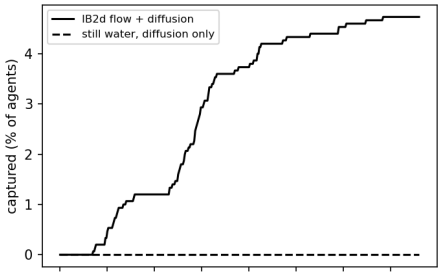
```
# starfish_only.vertex: the 405 starfish points of starfish_channel.vertex, without the walls
env.read_IB2d_mesh_data("starfish_only.vertex", method="adjacent",
                        add_idx_list=[(404, 0)])     # 0-based: last point to first
```

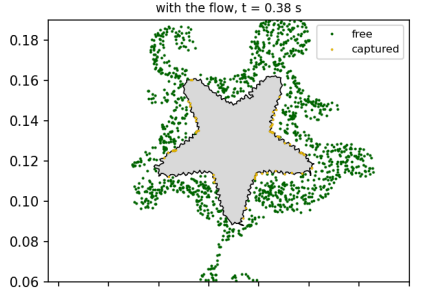
call	what it gave (405 points, 0.503–0.566 grid cells apart)
`method="adjacent", periodic=True`	404 segments: still open (`periodic` is read only for moving meshes)
`add_idx_list=[(405, 1)]` (1-based, as the docstring says)	`IndexError`: the indices are used from 0
`add_idx_list=[(404, 0)]`	405 segments, closed
`method="proximity"` (joins points closer than $0.501\,\Delta x$)	13 segments; 381 points joined to nothing
`method="proximity", res_factor=0.6`	442 segments: closed, plus 37 chords across

- The channel walls ($y = 0.01$ and 0.24) run across the box and cross its periodic x boundary, which Planktos does not handle (slide 34): load the starfish alone, and let `noflux` box edges stand in for the walls.
- Count the segments (`len(env.ibmesh)`) as well as looking at `plot_envir()`: a 0.57-cell gap (1.1 mm) does not show.

`examples/Planktos/starfish_env.py` (the `adjacent` call) and its `README.md` (each option tried on the same 405 points); `Environment.read_IB2d_mesh_data` (`planktos/_environment.py`, 1.0.3).

THE CAPTURE PATTERN ON THIS SERIES' STARFISH





- The `CaptureSwarm` of slide 35 (without its colour column), `ib_condition="sticky"`, the starfish of the previous slide: 1500 agents start within 1.5 cm of the outline (no closer than one grid cell), `cov` $= 2 \times 10^{-6}$ ($D = 10^{-6}$ m^2/s), $\Delta t = 1$ ms.
- With the flow 4.7 % are held by $t = 0.38$ s; in still water (the same environment with `flow` set to zeros, the same seed) none. Diffusion alone moves an agent about 0.9 mm in 0.38 s ($\sqrt{2Dt}$ in each direction): the flow brings them to the surface.

`examples/Planktos/starfish_capture.py` (Planktos 1.0.3, 90 s for both runs).

IMPORT MATRIX: CFD FIELDS AND 3D GEOMETRY

Source	Current call pattern	Audit item
IBAMR 3D VTK	`env.read_IBAMR3d_vtk_data(path, d_start=0, d_finish=None, vel_conv=None, grid_conv=None)`	dump range; spatial and velocity conversions
COMSOL VTU	`env.read_comsol_vtu_data(file, vel_conv=None, grid_conv=None)`	cell-centered versus nodal grid; units
NetCDF	`env.load_NetCDF(file)` then `env.read_NetCDF_flow("u","v", time_name="time", dim_reorder=...)`	variable names, dimension order, component axis, calendar/time units
STL mesh	`env.read_stl_mesh_data(file, unit_conv=None)`	mesh and flow share the same coordinate frame
IB2d mesh	`env.read_IB2d_mesh_data(file, method="adjacent"|"proximity"|"hull")`	breaks/additional edges; closed geometry

> The loader may reset `env.L` to match the data. Read back `L`, `flow_points`, `flow_times`, `fluid_domain_LLC`, and units after every import.

Signatures audited against `planktos/_environment.py`, v1.0.3.

ALIGN FLOW, MESH, AND PHYSICAL UNITS BEFORE RELEASE

```
# For appropriate cell-centered COMSOL data,
# regrid immediately after the flow import.
# Match these flags to the data's actual BCs;
# this example treats all axes as nonperiodic (the
      repository's
# sea-fan example passes (True, True, True) for its COMSOL
      data).
env.center_cell_regrid(
    periodic_dim=(False, False, False))

# Only then load the immersed geometry.
env.read_stl_mesh_data("geometry.stl")

print("domain:", env.L, env.units)
print("flow origin:", env.fluid_domain_LLC)
print("flow axes:", [p[[0, -1]] for p in env.flow_points])
print("mesh shape:", None if env.ibmesh is None
      else env.ibmesh.shape)

env.plot_envir()
env.plot_flow()
```

One consistent conversion ledger

Record conversions for length, velocity, time, density, viscosity, gravity, particle diameter, and mesh coordinates. Prefer converting on import with `vel_conv`, `grid_conv`, or `unit_conv`.

Visual mesh audit

Look for crossing edges, missing closure, duplicated surfaces, mesh outside the flow domain, and unintended contact with a periodic face.

Pass the flags explicitly: in v1.0.3 the `center_cell_regrid` docstring states a default of `(True, True, False)`, but the signature default is `(False, False, False)`. The regrid does not shift meshes already loaded, hence “only then”.

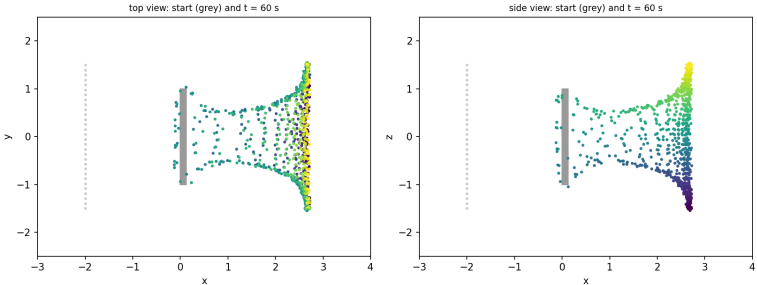
FROM AN IBAMR RUN TO PLANKTOS (1/2): THE ROUTE

```
# 1. VisIt (chapter 33): the AMR data on a uniform grid, as legacy VTK (here IBFE/10's last dump, around the
     plate)
visit -nowin -cli -s resample_export.py -- viz_IB3d/dumps.visit U plate_visit \
      nx=97 ny=49 nz=49 x0=-4 x1=8 y0=-3 y1=3 z0=-3 z1=3 states=20 format=binary
# 2. the velocity made the active vector; files named IBAMR_db_000.vtk, IBAMR_db_001.vtk, ...
python vtk_for_planktos.py plate_visit plate_planktos
# 3. Planktos: one file is a steady flow; a folder of IBAMR_db_###.vtk files, a time series
env = planktos.Environment(rho=1.0, mu=0.01)
env.read_IBAMR3d_vtk_data("plate_planktos/IBAMR_db_000.vtk")
print(env.L, env.fluid_domain_LLC, env.flow_times)   # [12.0, 6.0, 6.0] (-4.0, -3.0, -3.0) None
```

- Step 2 because VisIt 3.5 exports the vector as a field array beside an active scalar, and Planktos's reader then stops ("AttributeError: 'NoneType' object has no attribute 'GetDataType"'). VTK 9.4 may also print "Could not read dimensions or extents from the file" for these binary files: the grid still comes out whole.
- In 2021 this needed a Python 2.7 environment with VisIt (slide 9); now VisIt's own Python 3 does step 1.
- Spacing 0.125, twice the finest level's (chapter 22): $97 \times 49 \times 49$ points, 7.5 MB. The whole $32 \times 16 \times 16$ tank at that spacing would be about 18 times as many points.

`wsl/final/ch35_plate_export.sh` (VisIt 3.5.0, 8 s); `examples/analysis/vtk_for_planktos.py`; `examples/Planktos/ibamr_plate.py`.

FROM AN IBAMR RUN TO PLANKTOS (2/2): AGENTS PAST IBFE/10'S PLATE



- The plate is its reference shape, a box $0 < x < 0.125$, $-1 < y, z < 1$, written as an STL file (12 triangles, numpy-stl) in the run's coordinates; `read_stl_mesh_data` shifts it with the flow (to $x = 4$–4.125). By $t = 0.5$ its flaps had bent 0.012 at their edges (chapter 22).
- 900 tracers, a 30×30 sheet at $x = -2$, with a little diffusion; 600 steps of 0.1 s: 86 agent-steps touched the plate (sliding). The sheet bends round the plate, the agents behind it lag in its wake, and 18 still sit against its upstream face.
- The last dump stands in for a steady flow; the stream was still speeding up ($U \tanh 2t$ is 76 % of U at $t = 0.5$), so a longer run would give a truer one.

`examples/Planktos/ibamr_plate.py` (Planktos 1.0.3, 40 s); IBFE/10 (chapter 22).

FTLE REVEALS FINITE-TIME STRETCHING OF A MOTION MODEL

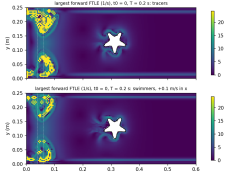
Let $\tau = T - t_0 > 0$. For the flow map $\mathbf{\Phi}_{t_0}^{T}$, the largest forward-time exponent is

$$\sigma_{t_0}^{\tau}(\boldsymbol{x}_0) = \frac{1}{\tau} \log \sqrt{\lambda_{\max}[(D\mathbf{\Phi})^T D\mathbf{\Phi}]}.$$

```
env.calculate_FTLE(
    grid_dim=(512, 128),
    t0=0, T=0.1, dt=0.001)
env.plot_2D_FTLE()

# Backward from physical time 0.1 to 0.0:
# current API uses T as the forward pseudo-time endpoint.
env.calculate_FTLE(
    (512, 128), t0=0.1, T=0.2,
    dt=0.001, backward=True)
```

Calls adapted from `examples/ex_produce_ftle_2d.py`.



The first call on this series' starfish flow (256×64 points, `T=0.2`, `testdir="x1"`, `dt=0.002`) for tracers (top) and, with `swrm=`, for swimmers that add 0.1 m/s in x (bottom): `examples/Planktos/starfish_ftle.py`, 140 s per map. Strickland et al. (2022), Fig. 5, compares FTLE fields for parasitoid wasps (not reproduced).

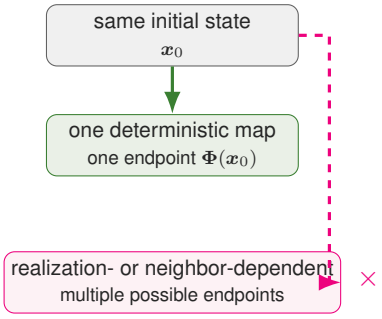
Two API distinctions

- The current call requires `T > t0`; the elapsed interval is `T-t0` (`T` is an end time, although the docstring calls it "integration time" and `plot_2D_FTLE`'s title prints it as Δt). With `backward=True`, physical time ends at `2*t0-T`; tracers only.
- `FTLE_smallest` is a forward-map contraction exponent, not backward-time FTLE.

WHEN AN AGENT-BASED FTLE FIELD IS MEANINGFUL

The `swrm=` route is appropriate when the seeded agents define one reproducible finite-time map:

- the motion rule is deterministic;
- agents use the same rule and parameter set;
- each trajectory acts independently of the other seeded agents;
- the map is resolved well enough in time and on the initial-condition grid.



Not an ordinary FTLE map

Stochastic realizations or social rules that depend on the seeded population do not define a single deterministic particle map. Use ensemble dispersion, encounter, or density diagnostics instead.

Qualification follows Strickland et al. (2022), Sections 2.1 and 4.2.

FTLE is a property of the chosen finite-time map, not of “complex-looking” trajectories in general.

Schematic: instructor’s course notes.

CAPSTONE: VICSEK AGENTS AROUND AN IMMERSED CYLINDER

```
import numpy as np
import planktos

env = planktos.Environment(
    Lx=1, Ly=0.25,
    x_bndry="periodic",
    y_bndry="periodic")
env.read_IB2d_mesh_data(
    "examples/vicsek_mesh/channel.vertex",
    method="proximity", res=0.001953125)

rng = np.random.default_rng(2026)
N = 500
IC = np.empty((N, 2))
IC[:, 0] = rng.uniform(0.075, 0.125, N)
IC[:, 1] = rng.uniform(0.050, 0.200, N)
```

Setup: current `examples/ex_vicsek_model_2d.py`; separate NumPy seed added.

Null versus flow experiment

The repository example runs without the large external flow dataset. Load matching IB2d flow to ask how alignment survives an unsteady wake: slide 49 does it with this series' starfish flow.

Strickland et al. (2022), Fig. 9, motivates the capstone with a 3D Vicsek result in cylinder flow (not reproduced: the article is not openly licensed).

CAPSTONE: IMPLEMENT ALIGNMENT WITH PERIODIC DISTANCES

```
class Vicsek(planktos.Swarm):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.shared_props.update(v=0.02, nu=0.5, r=0.1)
        angle = self.get_prop("nu") \
              * (self.rndState.random(self.N)-0.5)
        self.velocities += self.get_prop("v") \
              * np.column_stack((np.cos(angle), np.sin(angle)))

    def apply_agent_model(self, dt):
        theta = np.empty(self.N)
        L = np.asarray(self.envir.L)
        for i, origin in enumerate(self.positions):
            delta = (self.positions-origin + L/2) % L - L/2
            near = np.linalg.norm(delta, axis=1) < self.get_prop("r")
            theta[i] = np.arctan2(
                self.velocities[near, 1].mean(),
                self.velocities[near, 0].mean())
        theta += self.get_prop("nu")*(self.rndState.random(self.N)-0.5)
        active = self.get_prop("v")*np.column_stack((np.cos(theta),
     np.sin(theta)))
        return self.positions + (active + self.get_fluid_drift())*dt
```

Three model ingredients

1. average neighbor velocity vectors within radius `r`;
2. add angular noise of width `nu`;
3. add self-propulsion speed `v` and local fluid drift.

Periodic metric

The minimum-image displacement is essential: agents near opposite faces can still be neighbors. Note that the minimum-image wrap assumes periodic boundaries in both axes.

Condensed from the current repository Vicsek example (`examples/ex_vicsek_model_2d.py`); parameter names and hook are unchanged. Model: Vicsek et al. (1995).

CAPSTONE: WHAT THE ALIGNMENT RULE ACTUALLY AVERAGES

In the notation of Vicsek et al. (1995), each agent i updates its heading as

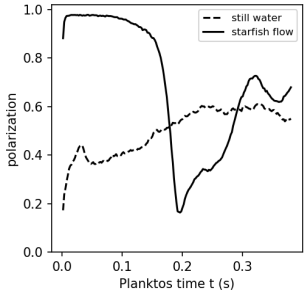
$$\theta_i^{n+1} = \arg\Big(\sum_{j:\,|\boldsymbol{x}_j - \boldsymbol{x}_i| < r} \boldsymbol{v}_j^{\,n} \Big) + \xi_i^n, \qquad \xi_i^n \sim \mathcal{U}\big[-\tfrac{\eta}{2}, \tfrac{\eta}{2}\big]\,,$$

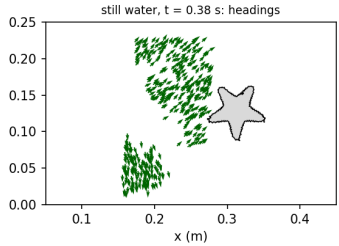
then moves with $\boldsymbol{x}_i^{n+1} = \boldsymbol{x}_i^n + \big[v\,(\cos\theta_i^{n+1}, \sin\theta_i^{n+1}) + \boldsymbol{u}(\boldsymbol{x}_i^n, t^n)\big]\Delta t$. Here η is `nu`, v is `v`, and r is `r`.

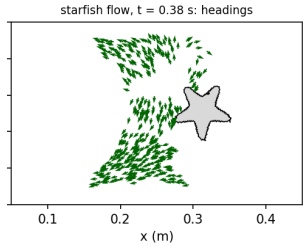
- Because `move` sets `velocities` from each step's displacement over `dt` (the proposed one, or the slide along a mesh), the neighbor average uses ground-frame velocities (swimming plus fluid drift), not swimming headings alone. In a strong flow the fluid can dominate the "consensus".
- An agent always counts itself as a neighbor (distance $0 < r$), so the average is never empty.
- The condensed listing wraps both axes unconditionally; the repository version wraps only the axes whose `envir.bndry` is `periodic`, so use that form in a channel with walls.
- The noise is drawn once per step, so the statistics depend on Δt; this is why the capstone asks for at least two smaller Δt.
- A standard summary is the polarization $\varphi = \big|\sum_i \boldsymbol{s}_i\big|/(Nv) \in [0, 1]$, with $\boldsymbol{s}_i$ the self-propulsion velocity; $\varphi \approx 1$ is an aligned flock.

Vicsek et al. (1995), Eqs. (1)–(2) and their order parameter; Planktos example `ex_vicsek_model_2d.py`, commit `a1c2128`.

CAPSTONE: THE RULE AROUND THIS SERIES' STARFISH







- The capstone's rule with all pairs at once (no loop), the minimum image on the periodic x axis only, and random starting headings: 400 agents released upstream of chapter 4f's starfish, $v = 0.1$ m/s, $r = 2$ cm, $\texttt{nu} = 0.5$, $\Delta t = 2$ ms, for the run's 0.38 s.
- Still water: the polarization grows from 0.17 to about 0.6 (0.55 at the end), and the swarm splits into two groups. The flow: 0.88 after the first step and 0.98 by $t = 0.02$ s, because the neighbours' mean velocity is the stream's (previous slide); briefly below 0.2 (0.16 at $t = 0.196$ s) as the stream near the agents dies away (its mean u from 0.31 m/s at $t = 0.08$ s to 0 at $t = 0.18$ s); 0.68 at the end.

`examples/Planktos/starfish_vicsek.py` (Planktos 1.0.3, 25 s for both runs).

RUN THE CAPSTONE, THEN MAKE THE RESULT REPRODUCIBLE

```
s = Vicsek(
    swarm_size=N, envir=env, init=IC, seed=2026)

collision_events = 0
for _ in range(72):        # 18 s at dt=0.25 s
    s.move(0.25, silent=True)
    collision_events += np.count_nonzero(
        s.ib_collision_idx >= 0)

print("agent-step collisions:", collision_events)

s.plot_all(
    movie_filename="vicsek_null.mp4",
    fps=4, fluid="vort")
```

Capstone comparisons

Repeat with a passive swarm, multiple `nu/r` values, and at least two smaller dt. Compare polarization, retention, agent-step collisions, and downstream spread.

Save with every result

- Planktos version/commit and Python/package environment;
- flow and mesh provenance plus checksums;
- both random seeds and all model parameters;
- time step, duration, collision mode, and active-agent count;
- code, raw histories, and plotting settings.

Failure-state diagnostic

If a collision step fails, current Planktos marks `env.time=None`. Inspect the partial state; then restore consistently from histories or restart from a checkpoint.

THIS SERIES' PLANKTOS EXAMPLES

examples/Planktos/	what it does	slides
brinkman_first.py	most of this chapter's first-run fragments, in order: Brinkman flow, a swarm, 24 s, plots, save_data, CSV, VTK (a movie if FFmpeg is found; not run here, without FFmpeg); no input data	25
starfish_env.py	load(run): chapter 4f's starfish run as an Environment, the starfish as a closed mesh	37, 38
starfish_kde.py	agents upstream of the starfish; vorticity and KDEs	26
starfish_capture.py	the capture pattern, with the flow and in still water	39
starfish_ftle.py	FTLE maps for tracers and for swimmers	44
starfish_vicsek.py	the capstone's rule in still water and in the flow	49
ibamr_plate.py	IBFE/10's flow tank through VisIt; a sheet of agents past the plate	42, 43

Run on Planktos 1.0.3 (git a1c2128) with Python 3.12 (Anaconda, Windows) on a desktop PC; the folder's README.md gives each script's time and result. The starfish run is chapter 4f's example (pyIB2d), IBFE/10 chapter 22's.

examples/Planktos/README.md.

KNOWN ISSUES IN 1.0.3, AND WHAT THIS CHAPTER DOES ABOUT THEM (1/2)

issue (October 2026)	what to do
the `nu` and `rho` setters compute $\mu = \nu/\rho$ (slide 32)	give `rho` and `mu` (or `nu`) to the constructor
`center_cell_regrid`: docstring default `(True, True, False)`, signature `(False, False, False)` (slide 41)	pass `periodic_dim` explicitly
`read_IB2d_mesh_data` on a `.vertex` file with `'adjacent'`: `periodic=True` ignored; `add_idx_list` used from 0, documented from 1 (slide 38)	close an outline with `add_idx_list=[(n-1, 0)]`; count the segments
`read_IB2d_fluid_data`: the docstring lists an `auto_regrid` parameter the signature does not have	nothing: the periodic end points are always added
`calculate_FTLE`: the docstring speaks of an `ode` argument (it is `ode_gen`) and says "Default is 1" for `T`, whose default is 0.1; `T` is an end time (slide 44)	trust the signature: `inspect.signature(env.calculate_FTLE)`

The pyIB2d item, four more traps of 1.0.3 met while running this chapter's fragments, and the search of Planktos's issues are on the next slide.

`planktos/_environment.py` (1.0.3); `examples/Planktos`.

KNOWN ISSUES IN 1.0.3, AND WHAT THIS CHAPTER DOES ABOUT THEM (2/2)

issue (October 2026)	what to do
pyIB2d writes dump 0's velocity transposed (IB2d issue #12; slide 37)	read from dump 1
`Swarm.plot_all(fluid="vort")` on a static flow, such as the Brinkman layer (slide 27), stops with "ValueError: not enough values to unpack (expected 2, got 1)" (`_swarm.py`, line 2548), with or without a movie file	`fluid="quiver"`, or none, on a static flow; `"vort"` works on a time-dependent one
`init="grid"` with `testdir` and no immersed mesh stops with "AttributeError: 'NoneType' object has no attribute 'reshape'" (slide 20)	give `testdir` only when a mesh is loaded
a `'D'` property is read only when there is no `'cov'`, and every Swarm starts with `cov` = the identity unless given one (slide 22)	set `'cov'` to $2D$, or delete `'cov'` before setting `'D'`
`set_brinkman_flow`: the docstring calls `alpha=0` free flow; the call divides by α^2 and stops with "ZeroDivisionError: float division by zero" (slide 16)	a small `alpha` instead (10^{-6} gives the free-flow profile)

Searching Planktos's issues on 17 October 2026 (`add_idx_list`, `periodic`, `regrid`, `setter`, `viscosity`, `docstring`, `vort`, `plot_all`, `testdir`, `diffusion`, `brinkman`) found none of the Planktos items on these two slides; related open ones are #75 (the 3D `.vertex` reader, chapter 34), #33 (moving the SDE solver to Milstein) and #74 (both fluid backdrops at once in `plot` and `plot_all`).

`planktos/_environment.py`, `_swarm.py` and `motion.py` (1.0.3); `examples/Planktos`; https://github.com/mountaindust/Planktos/issues.

RESOURCES AND SOURCE TRAIL

Start with the current software

- Planktos repository and README;
- repository `docs/` and runnable `examples/`;
- `examples/ib2d_data/README.txt` for the external IB2d dataset;
- this tutorial was audited against v1.0.3, commit `a1c2128` (2026-08-19);
- this series' `examples/Planktos` (its README: what each script gave);
- the lab's 2021 deck, "Getting started with Planktos" (install, tests, examples of 2021).

Record the exact software commit with every scientific result: current interfaces can differ from those shown in the 2022 paper.

Primary scientific sources

Strickland et al. (2022), *Bulletin of Mathematical Biology* 84:72

Software architecture, agent models, collision handling, visualization, FTLE, and examples.

Hamlet et al. (2023), *Journal of Experimental Biology* 226, jeb244520

Gorgonian multiscale flow and capture application; article and reproduced figures are CC BY 4.0.

REFERENCES

- W. C. Strickland, N. A. Battista, C. L. Hamlet, and L. A. Miller, “Planktos: an agent-based modeling framework for small organism movement and dispersal in a fluid environment with immersed structures,” *Bull. Math. Biol.* 84 (2022), 72, doi:10.1007/s11538-022-01027-1. Its Figs. 4, 5 and 9 are described, not reproduced (the article is not openly licensed): the pictures are this series’ runs. FTLE qualifications from Sections 2.1 and 4.2.
- C. L. Hamlet, W. C. Strickland, N. Battista, and L. A. Miller, “Multiscale flow between the branches and polyps of gorgonians,” *J. Exp. Biol.* 226(5) (2023), jeb244520, doi:10.1242/jeb.244520. CC BY 4.0; Figs. 1 and 6A reproduced.
- W. C. Strickland *et al.*, *Planktos* agent-based modeling framework, v1.0.3, commit `a1c2128` (2026-08-19), GPLv3, https://github.com/mountaindust/Planktos; documentation https://planktos.readthedocs.io. Source of every code pattern (`README.md`, `setup.cfg`, `examples/`, docstrings) and of the `docs/_static` images.
- S. Kupfner Johnson and P. Hallock, “A review of symbiotic gorgonian research in the western Atlantic and Caribbean with recommendations for future work,” *Coral Reefs* 39 (2020), 239–258, doi:10.1007/s00338-020-01891-0. Adapted in Hamlet et al. (2023), Fig. 1.
- M. R. Maxey and J. J. Riley, “Equation of motion for a small rigid sphere in a nonuniform flow,” *Phys. Fluids* 26 (1983), 883–889, doi:10.1063/1.864230.
- G. Haller and T. Sapsis, “Where do inertial particles go in fluid flows?” *Physica D* 237 (2008), 573–583, doi:10.1016/j.physd.2007.09.027. Linearized form used by `inertial_particles`.

REFERENCES (CONTINUED)

- T. Vicsek, A. Czirók, E. Ben-Jacob, I. Cohen, and O. Shochet, “Novel type of phase transition in a system of self-driven particles,” *Phys. Rev. Lett.* 75 (1995), 1226–1229, doi:10.1103/PhysRevLett.75.1226.
- S. C. Shadden, F. Lekien, and J. E. Marsden, “Definition and properties of Lagrangian coherent structures from finite-time Lyapunov exponents in two-dimensional aperiodic flows,” *Physica D* 212 (2005), 271–304, doi:10.1016/j.physd.2005.10.007.
- G. Haller, “Lagrangian coherent structures,” *Annu. Rev. Fluid Mech.* 47 (2015), 137–162, doi:10.1146/annurev-fluid-010313-141322.
- P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*. Springer, 1992, doi:10.1007/978-3-662-12616-5. Euler–Maruyama (§10.2) and Milstein (§10.3) schemes.
- J. R. Dormand and P. J. Prince, “A family of embedded Runge–Kutta formulae,” *J. Comput. Appl. Math.* 6 (1980), 19–26, doi:10.1016/0771-050X(80)90013-3. The RK45 pair behind `motion.RK45`.
- H. C. Brinkman, “A calculation of the viscous force exerted by a flowing fluid on a dense swarm of particles,” *Appl. Sci. Res.* A1 (1949), 27–34, doi:10.1007/BF02120313.

REFERENCES (CONTINUED)

- W. C. Strickland, L. A. Miller, A. Santhanakrishnan, C. Hamlet, N. A. Battista, and V. Pasour, “Three-dimensional low Reynolds number flows near biological filtering and protective layers,” *Fluids* 2 (2017), 62, doi:10.3390/fluids2040062. Analytic Brinkman profile behind `set_brinkman_flow`.
- D. W. Scott, *Multivariate Density Estimation*. Wiley, 1992, doi:10.1002/9780470316849. Scott’s KDE bandwidth factor (`dist="density"`).
- N. A. Battista, W. C. Strickland, and L. A. Miller, “IB2d: a Python and MATLAB implementation of the immersed boundary method,” *Bioinspir. Biomim.* 12 (2017), 036003, doi:10.1088/1748-3190/aa5e08. Source of the IB2d flow and mesh formats.
- FFmpeg project, https://ffmpeg.org. External movie writer used by Matplotlib for `plot_all(movie_filename=...)`.

REFERENCES (CONTINUED)

- Planktos release notes v1.0.1–v1.0.3 and `README.md`, https://github.com/mountaindust/Planktos (read 17 October 2026); IB2d issue #12, https://github.com/nickabattista/IB2d/issues/12.
- “Getting started with Planktos” (Miller lab, 2021): the install, tests and example tour merged here, and the 2021 pictures.

> The workflow, object-model, Brinkman-layer, shared-clock, and FTLE-validity diagrams were drawn for these notes (TikZ). Code fragments are adapted from the Planktos repository (GPLv3) as credited on each slide; the plots are this series’ runs (`examples/Planktos`), the 2021 lab deck’s, and two Planktos documentation images (the title slide and slide 34).